

PRACTICA 3 - ALGORITMICA

En esta práctica vamos a realizar el ejercicio 4 de la hoja de ejercicios, que es la implementación en código C++ de los algoritmos de Dijkstra y Bellman-Ford para sus respectivos grafos de los ejercicios 1 y 3.

Antes de entrar al código, mencionar que con los grafos se pueden utilizar matrices de adyacencia y de incidencia, la primera es una buena opción, pero la segunda no es muy recomendable con estos algoritmos.

También hay una lista de adyacencia, muy usada en programación, en la que crearíamos un vector con los vértices, y claro estos vértices (podemos hacerlos estructuras perfectamente) contienen una lista de sus conexiones o aristas hacia otros vértices con sus pesos.

Pero yo he decidido usar una lista de Aristas, que es la que más sencilla me ha parecido, y la que mejor se puede implementar para usar ambos algoritmos. Es similar a la anterior, simplemente se basa en un vector de Aristas, que contienen un origen, un destino, y su peso.

Ahora vamos al código por partes, iremos de las funciones complementarias a las principales.

FUNCIONES COMPLEMENTARIAS

Aquí nos encontramos las bibliotecas usadas, la constante INF, y la estructura Arista que antes hemos explicado.

Después la función 'cargadoCorrectamente', que simplemente muestra el grafo una vez lo hemos creado, para ver que todo va correcto.

Y finalmente la función mostrar estado, que usan ambos algoritmos, y sirve para ver como se encuentra d y pred en cada paso, está en un bucle en ambos algoritmos.

```
#include <iostream>
#include <vector>
#include <string>
// #include <limits>
using namespace std;

const int INF = 999999;

struct Arista {
    int origen;
    int destino;
    int peso;
};

void cargadoCorrectamente(vector<string>& nombres, vector<Arista>& Aristas){

    cout << "Grafo cargado correctamente:" << endl;

    for (Arista arista : Aristas) {
```

```
        cout << nombres[arista.origen] << " -> " << nombres[arista.destino] << " =  
" << arista.peso << endl;  
    }  
  
}  
  
void mostrarEstado(vector<int>& d, vector<int>& pred, vector<string>& nombres) {  
  
    cout << "Vertice\t d\t pi" << endl;  
  
    for (int i = 0; i < d.size(); i++) {  
        cout << nombres[i] << "\t ";  
  
        if (d[i] == INF)  
            cout << "INF";  
        else  
            cout << d[i];  
  
        cout << "\t ";  
  
        if (pred[i] == -1)  
            cout << "-";  
        else  
            cout << nombres[pred[i]];  
  
        cout << endl;  
    }  
}
```

ALGORITMO BELLMAN-FORD

El algoritmo Bellman-Ford se utiliza para calcular los caminos mínimos desde un vértice origen hasta el resto de vértices del grafo, incluso cuando existen aristas con pesos negativos, algo que se ha añadido al final de la práctica, ya que algunos resultados podían ser engañosos o no estar correctos.

En mi implementación, primero se inicializan los vectores de distancias (d) y predecesores (pred). Después, mediante un bucle que se repite V-1 veces, se recorren todas las aristas del grafo actualizando las distancias cuando se encuentra un camino más corto. Finalmente, se realiza una comprobación adicional para detectar posibles ciclos negativos.

```
void bellmanFord(int V, vector<Arista>& aristas, int origen, vector<string>&  
nombres) {  
  
    vector<int> d(V, INF);  
    vector<int> pred(V, -1);
```

```

d[origen] = 0;

cout << "\nEstado inicial:" << endl;
mostrarEstado(d, pred, nombres);

for (int i = 1; i <= V - 1; i++) {
    cout << "\nPaso " << i << ":" << endl;

    for (Arista a : aristas) {
        if (d[a.origen] != INF && d[a.origen] + a.peso < d[a.destino]) {
            d[a.destino] = d[a.origen] + a.peso;
            pred[a.destino] = a.origen;
        }
    }

    mostrarEstado(d, pred, nombres);
}

for (Arista a : aristas) {
    if (d[a.origen] != INF && d[a.origen] + a.peso < d[a.destino]) {
        cout << "\nEl grafo contiene un ciclo negativo." << endl;
        return;
    }
}

cout << "\nNo hay ciclos negativos alcanzables desde el origen." << endl;
}

```

EJECUCION BELLMAN-FORD

===== BELLMAN-FORD - GRAFO EJERCICIO 1 - ORIGEN z =====

Grafo cargado correctamente:

```

s -> t = 6
s -> y = 7
t -> x = 5
t -> y = 8
t -> z = -4
x -> t = -2
y -> x = -3
y -> z = 9
z -> x = 7
z -> s = 2

```

Estado inicial:

Vertice	d	pi
s	INF	-
t	INF	-
x	INF	-
y	INF	-

z	0	-
---	---	---

Paso 1:

Vertice	d	pi
s	2	z
t	INF	-
x	7	z
y	INF	-
z	0	-

Paso 2:

Vertice	d	pi
s	2	z
t	5	x
x	6	y
y	9	s
z	0	-

Paso 3:

Vertice	d	pi
s	2	z
t	4	x
x	6	y
y	9	s
z	0	-

Paso 4:

Vertice	d	pi
s	2	z
t	4	x
x	6	y
y	9	s
z	0	-

No hay ciclos negativos alcanzables desde el origen.

===== BELLMAN-FORD - GRAFO EJERCICIO 1 MODIFICADO - ORIGEN s =====

Grafo cargado correctamente:

s -> t = 6
s -> y = 7
t -> x = 5
t -> y = 8
t -> z = -4
x -> t = -2
y -> x = -3
y -> z = 9
z -> x = 4
z -> s = 2

Estado inicial:

Vertice	d	pi
s	0	-
t	INF	-
x	INF	-

y	INF	-
z	INF	-

Paso 1:

Vertice	d	pi
s	0	-
t	6	s
x	4	y
y	7	s
z	2	t

Paso 2:

Vertice	d	pi
s	0	-
t	2	x
x	4	y
y	7	s
z	2	t

Paso 3:

Vertice	d	pi
s	0	-
t	2	x
x	2	z
y	7	s
z	-2	t

Paso 4:

Vertice	d	pi
s	0	-
t	0	x
x	2	z
y	7	s
z	-2	t

El grafo contiene un ciclo negativo.

ALGORITMO DIJKSTRA

El algoritmo de Dijkstra también permite calcular los caminos mínimos desde un vértice origen, aunque solo funciona correctamente con pesos no negativos.

En el código se inicializan las distancias, los predecesores y un vector de vértices visitados. En cada iteración se selecciona el vértice no visitado con menor distancia y se actualizan las distancias de sus vecinos si se encuentra un camino más corto. El proceso se repite hasta que todos los vértices alcanzables han sido procesados.

```
void dijkstra(int V, vector<Arista>& aristas, int origen, vector<string>& nombres)
{
```

```

vector<int> d(V, INF);
vector<int> pred(V, -1);
vector<bool> visitado(V, false);

d[origen] = 0;

cout << "Estado inicial:" << endl;
mostrarEstado(d, pred, nombres);

for (int paso = 1; paso <= V; paso++) {
    int u = -1;
    int menor = INF;

    // Buscar el vertice no visitado con menor distancia
    for (int i = 0; i < V; i++) {
        if (!visitado[i] && d[i] < menor) {
            menor = d[i];
            u = i;
        }
    }

    if (u == -1) {
        break;
    }

    visitado[u] = true;

    // Relajar las aristas que salen de u
    for (Arista a : aristas) {
        if (a.origen == u) {
            if (d[u] != INF && d[u] + a.peso < d[a.destino]) {
                d[a.destino] = d[u] + a.peso;
                pred[a.destino] = u;
            }
        }
    }

    cout << "\nPaso " << paso << " - se selecciona " << nombres[u] << ":" <<
endl;
    mostrarEstado(d, pred, nombres);
}
}

```

EJECUCIÓN DIJKSTRA

```

===== DIJKSTRA - GRAFO EJERCICIO 3 - ORIGEN s =====
Grafo cargado correctamente:
s -> t = 3

```

```
s -> y = 5
s -> z = 3
t -> x = 6
t -> y = 2
y -> t = 1
y -> x = 4
y -> z = 6
x -> z = 2
z -> x = 7
```

Estado inicial:

Vertice	d	pi
s	0	-
t	INF	-
x	INF	-
y	INF	-
z	INF	-

Paso 1 - se selecciona s:

Vertice	d	pi
s	0	-
t	3	s
x	INF	-
y	5	s
z	3	s

Paso 2 - se selecciona t:

Vertice	d	pi
s	0	-
t	3	s
x	9	t
y	5	s
z	3	s

Paso 3 - se selecciona z:

Vertice	d	pi
s	0	-
t	3	s
x	9	t
y	5	s
z	3	s

Paso 4 - se selecciona y:

Vertice	d	pi
s	0	-
t	3	s
x	9	t
y	5	s
z	3	s

Paso 5 - se selecciona x:

Vertice	d	pi
s	0	-
t	3	s
x	9	t

```

y      5      s
z      3      s

===== DIJKSTRA - GRAFO EJERCICIO 3 - ORIGEN z =====
Grafo cargado correctamente:
s -> t = 3
s -> y = 5
s -> z = 3
t -> x = 6
t -> y = 2
y -> t = 1
y -> x = 4
y -> z = 6
x -> z = 2
z -> x = 7
Estado inicial:
Vertice d      pi
s      INF     -
t      INF     -
x      INF     -
y      INF     -
z      0       -

```

Paso 1 - se selecciona z:

```

Vertice d      pi
s      INF     -
t      INF     -
x      7       z
y      INF     -
z      0       -

```

Paso 2 - se selecciona x:

```

Vertice d      pi
s      INF     -
t      INF     -
x      7       z
y      INF     -
z      0       -

```

MAIN

En el main simplemente creamos los Vertices y declaramos la cantidad de ellos, declaramos también la lista de Artistas, y ejecutamos sobre los grafos los algoritmos correspondientes.

```

int main(){
    const int V = 5;

    int s = 0, t = 1, x = 2, y = 3, z = 4;

```



```
vector<string> nombres = {"s", "t", "x", "y", "z"};

// Grafo del ejercicio 1
vector<Arista> grafo1 = {
    {s, t, 6},
    {s, y, 7},
    {t, x, 5},
    {t, y, 8},
    {t, z, -4},
    {x, t, -2},
    {y, x, -3},
    {y, z, 9},
    {z, x, 7},
    {z, s, 2}
};

cout << "\n===== BELLMAN-FORD - GRAFO EJERCICIO 1 - ORIGEN z =====\n";
cargadoCorrectamente(nombres, grafo1);
bellmanFord(V, grafo1, z, nombres);

// Grafo del ejercicio 1 modificado: z -> x cambia de 7 a 4
vector<Arista> grafo1Modificado = {
    {s, t, 6},
    {s, y, 7},
    {t, x, 5},
    {t, y, 8},
    {t, z, -4},
    {x, t, -2},
    {y, x, -3},
    {y, z, 9},
    {z, x, 4},
    {z, s, 2}
};

cout << "\n===== BELLMAN-FORD - GRAFO EJERCICIO 1 MODIFICADO - ORIGEN s =====\n";
cargadoCorrectamente(nombres, grafo1Modificado);
bellmanFord(V, grafo1Modificado, s, nombres);

// Grafo del ejercicio 3
vector<Arista> grafo3 = {
    {s, t, 3},
    {s, y, 5},
    {s, z, 3},
    {t, x, 6},
    {t, y, 2},
    {y, t, 1},
    {y, x, 4},
    {y, z, 6},
    {x, z, 2},
    {z, x, 7}
};
```

```
cout << "\n===== DIJKSTRA - GRAFO EJERCICIO 3 - ORIGEN s =====\n";
cargadoCorrectamente(nombres, grafo3);
dijkstra(V, grafo3, s, nombres);

cout << "\n===== DIJKSTRA - GRAFO EJERCICIO 3 - ORIGEN z =====\n";
cargadoCorrectamente(nombres, grafo3);
dijkstra(V, grafo3, z, nombres);

return 0;
}
```